CSE311 OPERATING SYSTEMS

Pintos Project 2-2 Design Document

```
             +--------------------------+
             |          CS 140          |
             | PROJECT 2: USER PROGRAMS |
             |     DESIGN DOCUMENT      |
             +--------------------------+
```

---- GROUP ----

>> Fill in the names and email addresses of your group members.

Jeonghoon Park <hoonably@unist.ac.kr>
Deokhyeon Kim <ejrgus1404@unist.ac.kr>

---- PRELIMINARIES ----

>> If you have any preliminary comments on your submission, notes for the
>> TAs, or extra credit, please give them here.

>> Please cite any offline or online sources you consulted while
>> preparing your submission, other than the Pintos documentation, course
>> text, lecture notes, and course staff.

ARGUMENT PASSING

---- DATA STRUCTURES ----

>> A1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

- Nothing added.

---- ALGORITHMS ----

>> A2: Briefly describe how you implemented argument parsing. How do
>> you arrange for the elements of argv[] to be in the right order?
>> How do you avoid overflowing the stack page?

- I modified process_execute function so that it passes only the program name to the thread_create function not the whole command including arguments.
- I added some code in the process_start function to parse arguments and push them in the stack.
  Below is the brief process of that code.
  1. Divide input command into a number of token seperated by blanks, then store them in char * array.
  2. Push each arguments into the stack from right to left.
  3. Push padding (if required) to ensures that esp is multiples of 4.
  4. Push NULL sentinal since argv[argc] == NULL.
  5. Push address (in the stack) of each arguments from right to left.
  6. Push address (in the stack) of the first argument's address.
  7. Push argc.
  8. Push return address.

---- RATIONALE ----

>> A3: Why does Pintos implement strtok_r() but not strtok()?

- Since strtok() uses static variable in the internal implementation, it is unsafe in the multi-thread environment.
  However, since strtok_r() uses external variable instead of static variable it is safe in multi-thread environment.

>> A4: In Pintos, the kernel separates commands into a executable name
>> and arguments. In Unix-like systems, the shell does this
>> separation. Identify at least two advantages of the Unix approach.

- Since kernel doesn't have to consider parsing, kernel code is must simple.
- Kernel is much secure because program bugs related to parsing is restricted to user space (shell).
- Shell is more flexible, we can make many variations of shells to satisfy user requirements.

SYSTEM CALLS

---- DATA STRUCTURES ----

>> B1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

    - struct lock file_lock; → Global lock for controlling concurrent file system access
    - typedef int pid_t; → process ID type for exec and wait
    - int fd_idx; → next file discripter index
    - struct file *fd_table[FD_MAX]; → file discripter table (FD_MAX=128 setting now)
    - struct thread *parent; → Links child thread to its parent, enabling wait() and resource management.
    - struct list_elem child; → Embeds thread in parent's child_list, allowing retrieval during wait().
    - struct list child_list; → Stores all child threads for lookup and cleanup in wait() or parent exit.

    - struct semaphore s_load; → Synchronizes exec(): parent blocks until child finishes load, ensuring correct load result handling.
    - struct semaphore s_wait; → Used by parent to block until child exits, enabling correct wait() semantics.
    - bool is_wait; → Prevents duplicate wait() calls on the same child by tracking if wait was already called.
    - bool is_load; → Stores whether load() succeeded, enabling exec() to return -1 on failure.
    - int is_exit; → Holds the exit status code for this thread, returned to parent by wait().
    - struct file *cur_file; → Points to currently executing file; used to deny write access during execution.

>> B2: Describe how file descriptors are associated with open files.
>> Are file descriptors unique within the entire OS or just within a
>> single process?

- Each thread (or process) manages its own file descriptors using a local fd_table array.
- File descriptors are not global. The descriptor number is valid only within the process and is independent of other processes.

---- ALGORITHMS ----

>> B3: Describe your code for reading and writing user data from the
>> kernel.

    Exception Handling:
        - We use "get_user()" and "put_user()" functions to safely access user memory from the kernel.
        - These functions first validate the address using "is_valid_user_ptr()",
          which checks that the address is below "PHYS_BASE" and mapped in the page table.
        - For buffers (e.g., in read, write), we validate the entire region with "is_valid_buffer()"
          to avoid partial faults. Invalid pointers result in "exit(-1)" to terminate the offending process safely.
        - This prevents kernel panics caused by invalid memory accesses.
    Reading:
        - If fd (file descriptor) is 0, then read characters to buffer from console.
        - If fd is invalid or the file indicated by fd is invalid, then return -1.
        - Read contents of the file. To avoid race condition, we utilize lock here.
    Writing:
        - If fd (file descriptor) is 1, then print characters in buffer at console.
        - If fd is invalid or the file indicated by fd is invalid, then return -1.
        - Write contents to the file. To avoid race condition, we utilize lock here.

>> B4: Suppose a system call causes a full page (4,096 bytes) of data
>> to be copied from user space into the kernel. What is the least
>> and the greatest possible number of inspections of the page table
>> (e.g. calls to pagedir_get_page()) that might result? What about
>> for a system call that only copies 2 bytes of data? Is there room
>> for improvement in these numbers, and how much?

    - When copying 4096 bytes, the minimum number of pagedir_get_page() calls is 1 if all bytes are in the same page.
    - The maximum is 4096 if each byte is checked individually.
    - For 2 bytes, minimum is 1, maximum is 2 if they cross page boundaries.
    - Our current implementation checks per byte. resulting in up to one pagedir_get_page() call per byte.
      It is functionally correct but inefficient for large buffers.
    - It could be improved by checking per-page using page-aligned ranges,
      reducing lookups from O(n) to O(n / 4096).
      It would result in significantly improved performance for system calls like read() and write() that operate on large contiguous buffers.

>> B5: Briefly describe your implementation of the "wait" system call
>> and how it interacts with process termination.

    - In project 2-2, we replaced the temporary timer_msleep() used in 2-1 with a full implementation of process_wait().
    - Now, the parent process actively tracks the termination of a specific child by searching its child_list using get_child().
    - If the child exists and hasn't been waited on (is_wait == false), the parent blocks by calling sema_down(&s_wait).
    - When the child process exits, it calls sema_up(&s_wait) in process_exit(), waking up the parent.
    - The exit status is passed via is_exit and returned to the parent.
    - The parent then recives the exit status of child and marks the child as waited (is_wait = true) to prevent duplicate waits.

    - If the child does not exist, or has already been waited on, process_wait() returns -1.
    - This approach ensures proper synchronization and resource cleanup, avoiding busy-waiting and supporting correct wait semantics.

>> B6: Any access to user program memory at a user-specified address
>> can fail due to a bad pointer value. Such accesses must cause the
>> process to be terminated. System calls are fraught with such
>> accesses, e.g. a "write" system call requires reading the system
>> call number from the user stack, then each of the call's three
>> arguments, then an arbitrary amount of user memory, and any of
>> these can fail at any point. This poses a design and
>> error-handling problem: how do you best avoid obscuring the primary
>> function of code in a morass of error-handling? Furthermore, when
>> an error is detected, how do you ensure that all temporarily
>> allocated resources (locks, buffers, etc.) are freed? In a few
>> paragraphs, describe the strategy or strategies you adopted for
>> managing these issues. Give an example.

    - To safely access user memory, we use two helper functions: `is_valid_user_ptr()` for single addresses and `is_valid_buffer()` for ranges.
    - These are called before dereferencing user pointers, e.g., in `syscall_handler()`, `read()`, and `write()`.
    - If a bad pointer is detected, we immediately call `exit(-1)` to safely terminate the process and avoid kernel panic.
    - This centralized validation keeps syscall logic clean and avoids scattered checks.
    - temporarily acquired resources, such as file locks and file descriptors are released appropriately.
      File locks are acquired and released in each operation to avoid leaks or deadlocks.
      Remaining open files are released in `process_exit()`.
    - For example, `read(fd, buffer, size)` checks `is_valid_buffer()` first. On failure, it exits early, before any file operation.
    - Above described early exit and centralized validation ensures both robustness and clarity of our system call implemention.

---- SYNCHRONIZATION ----

>> B7: The "exec" system call returns -1 if loading the new executable
>> fails, so it cannot return before the new executable has completed
>> loading. How does your code ensure this? How is the load
>> success/failure status passed back to the thread that calls "exec"?

    - To synchronize `exec()`, we use a semaphore `s_load` in the child thread.
    - The parent calls `sema_down(&child->s_load)` after creating the child via `process_execute()`.
    - In the child's `start_process()`, once `load()` completes, the result is stored in `is_load`, and `sema_up(&s_load)` wakes the parent.
    - The parent resumes and checks `is_load`.
    - If loading failed, `exec()` returns -1; otherwise, it returns the child's exit status.
      This guarantees that `exec()` returns only after the child finishes loading,
      and that the return value reflects success or failure.
    - This design ensures that exec() avoids any race condition.

>> B8: Consider parent process P with child process C. How do you
>> ensure proper synchronization and avoid race conditions when P
>> calls wait(C) before C exits? After C exits? How do you ensure
>> that all resources are freed in each case? How about when P
>> terminates without waiting, before C exits? After C exits? Are
>> there any special cases?

    - When parent P calls wait(C) before child C exits:
    We use a semaphore s_wait to synchronize between the parent and child.
    If the parent calls wait(C) before the child exits, it blocks by calling sema_down(&child->s_wait).
    When the child later calls exit(), it signals the parent by invoking sema_up(&s_wait),
    allowing the parent to resume and retrieve the child's exit status via is_exit.
    The is_wait flag is then set to true to prevent duplicate waits, and the child is removed from the child_list.

    - When parent P calls wait(C) after child C has already exited:
    If the child exits before the parent calls wait(C), it stores its exit status in is_exit and signals via s_wait.
    When the parent eventually calls wait(C), it immediately returns with the stored status since the semaphore has already been signaled.
    is_wait is set to prevent future waits, and the child is removed from the list.

    - Ensuring that all resources are freed in each case:
    All open files in the child are closed in process_exit(), including the running executable via file_close(cur_file).
    If the parent successfully calls wait(C), the child's memory (its struct thread) is freed in thread_schedule_tail() when is_wait == true.
    This guarantees that both file descriptors and thread memory are released properly.

    - When parent P terminates without waiting, before child C exits:
    If the parent exits before calling wait(C), the child still runs and eventually calls exit() on its own.
    However, since is_wait remains false, the child's thread structure is never freed, resulting in a memory leak.
    This behavior is permitted under Project 2's scope, but would require additional mechanisms like reference counting in

a full OS.

    - When parent P terminates without waiting, after child C exits:
    Similar to case 4, if the child has already exited and the parent never calls wait(),
    the child's exit status remains stored, but is_wait is never set.
    Thus, the child's memory is not freed and leaks.
    Again, this case is allowed in Pintos but could be addressed more thoroughly in later projects.

    - Special cases and protection:
    Multiple wait() calls on the same child are prevented via is_wait.
    get_child() ensures only direct children can be waited on. grandchild processes are not eligible.
    If the child does not exist or has already been waited on, process_wait() returns -1.

---- RATIONALE ----

>> B9: Why did you choose to implement access to user memory from the
>> kernel in the way that you did?

    - We implemented user memory access checks using `is_valid_user_ptr()` and `is_valid_buffer()` to ensure both safety and clarity.
    - is_valid_user_ptr() verifies that a single user-provided pointer is non-null, lies within the user address space, and is mapped to a valid physical page.
    - is_valid_buffer() extends this by validating an entire memory range by one byte at a time,
      ensuring every address in the buffer is user-accessible and mapped to a valid physical page.
    - This avoids undefined behavior from invalid memory and protects the kernel from user-induced faults.
    - We chose to validate pointers *before* any dereference to keep syscall logic clean and centralized.
    - Invalid pointers result in `exit(-1)`, which safely terminates the process without risking kernel panic.
    - This design separates validation from core logic, improving readability and maintainability.
    - It also fails fast on invalid input, which is preferable to scattered try-catch-style handling in C.

>> B10: What advantages or disadvantages can you see to your design
>> for file descriptors?

    - We implemented file descriptors using a statically allocated array `fd_table[FD_MAX]` and an integer `fd_idx` to track the next available slot.
    - This design is simple and fast: file access is fast via array indexing, and implementation complexity is low.
    - However, it has limitations. The size is fixed, so a process cannot open more than `FD_MAX` files.
    - Also, `fd_idx` increases monotonically and doesn't reuse closed slots, which may lead to fragmentation or early exhaustion.
    - This design is sufficient for Pintos, but not ideal for systems requiring scalability or file descriptor recycling.

>> B11: The default tid_t to pid_t mapping is the identity mapping.
>> If you changed it, what advantages are there to your approach?

    - We kept the default identity mapping between `tid_t` and `pid_t`, as it was sufficient for all required operations.
    - Since each thread has a unique `tid`, and we only needed basic parent-child process tracking,
      introducing a separate mapping would have added unnecessary complexity without benefit.
    - If we had changed the mapping (e.g., to avoid PID reuse or support real process group IDs),
      it could improve abstraction and security. But for Pintos, identity mapping is simple, efficient, and enough.

               SURVEY QUESTIONS
               ================

Answering these questions is optional, but it will help us improve the
course in future quarters. Feel free to tell us anything you
want--these questions are just to spur your thoughts. You may also
choose to respond anonymously in the course evaluations at the end of
the quarter.

>> In your opinion, was this assignment, or any one of the three problems
>> in it, too easy or too hard? Did it take too long or too little time?

>> Did you find that working on a particular part of the assignment gave
>> you greater insight into some aspect of OS design?

>> Is there some particular fact or hint we should give students in
>> future quarters to help them solve the problems? Conversely, did you
>> find any of our guidance to be misleading?

>> Do you have any suggestions for the TAs to more effectively assist
>> students, either for future quarters or the remaining projects?

>> Any other comments?

==== Peer Evaluation ====

+--------------------------+---------------------------------+

Name	Contribution
Jeonghoon Park	50%, Implemented read, write, exec, wait, filesize / exception.c / pass multi-oom
Deokhyeon Kim	50%, Implemented remove, seek, tell / Finalization and fixes the previous code part